Random Forest for Regression

Random Forest एक **Ensemble Learning** method है।

👉 Ensemble का मतलब:

> कई models (यहाँ Decision Trees) को मिलाकर एक strong model बनाना।

👉 Simple भाषा में:

> Random Forest = बहुत सारे **Decision Trees** का समूह (forest 🌳🌳🌳)

हर tree अपना prediction देता है, और अंत में:

- **Classification** में → Voting (जो class ज्यादा vote पाए)
- **Regression** में → Average (सबका average)

Steps

Step 1: Bootstrap Sampling (Bagging)

- Dataset से random sampling होती है (with replacement)
- हर tree को अलग-अलग data मिलता है

👉 Example:

अगर 100 rows हैं → हर tree को 100 rows मिलेंगी, लेकिन कुछ repeat होंगी

Steps

Step 2: Random Feature Selection

- हर split पर सभी features नहीं देखे जाते
- केवल random features का subset लिया जाता है

👉 क्यों?

👉 ताकि trees अलग-अलग बनें (diversity बढ़े)

Step 3: Decision Trees बनते हैं

- हर tree independently train होता है

Step 4: Final Prediction

📌 **Classification:**

Majority Voting

📌 **Regression:**

Mean (average)

Random Forest in Both

📍 Regression vs Classification में Use

📊 Regression (continuous output)

- Example: House price prediction
- Output: number

👉 Random Forest:

➡️ सभी trees का average

🏷️ Classification (categorical output)

- Example: spam / not spam
- Output: class

👉 Random Forest:

➡️ Majority voting

Random Forest क्यों use करते हैं?

👉 क्योंकि single **Decision Tree** overfitting करता है

Random Forest:

- Variance कम करता है
- Stability बढ़ाता है
- Accuracy improve करता है

Characteristics

⚙️ Characteristics (विशेषताएँ)

- Ensemble method
- Bagging technique use करता है
- Random feature selection
- Multiple trees
- Parallel training possible

✅ Advantages

- ✔ Overfitting कम करता है
- ✔ High accuracy
- ✔ Noise handle कर सकता है
- ✔ Feature importance देता है
- ✔ Missing values को handle कर सकता है

❌ Disadvantages

- ❌ Model heavy होता है
- ❌ Slow हो सकता है
- ❌ Interpret करना मुश्किल
- ❌ Memory ज्यादा लेता है

Formula

अगर हमारे पास models हैं:

$h_1(x), h_2(x), h_3(x), \ldots, h_n(x)$

📌 **Regression:**

$$\hat{y} = \frac{1}{n} \sum_{i=1}^{n} h_i(x)$$

👉 मतलब: सभी trees का average

📌 **Classification:**

$$\hat{y} = \text{mode}(h_1(x), h_2(x), \ldots, h_n(x))$$

👉 मतलब: जो class सबसे ज्यादा बार आए

Formula

$h_1(x), h_2(x), \ldots h_n(x)$ का मतलब क्या है?

👉 यहाँ:

- **x** = आपका input data (जैसे किसी student की details, house features, etc.)
- **$h_1, h_2, h_3 \ldots h_n$** = अलग-अलग Decision Trees

👉 मतलब:

> हर tree (model) एक function है जो input x लेकर output देता है

📌 Example:

मान लो आपने 3 trees बनाए:

- $h_1(x) \rightarrow 100000$ (price predict)
- $h_2(x) \rightarrow 120000$
- $h_3(x) \rightarrow 110000$

Formula

Regression Formula का मतलब

$$\hat{y} = \frac{1}{n}\sum_{i=1}^{n} h_i(x)$$

👉 इसे आसान भाषा में:

- n = total trees
- $h_i(x)$ = हर tree का prediction
- ∑ = सबको जोड़ना

👉 Final output:

> सभी trees का **average (mean)**

Example:

3 trees हैं:

- $h_1(x) = 100$
- $h_2(x) = 120$
- $h_3(x) = 110$

$$\hat{y} = \frac{100 + 120 + 110}{3} = 110$$

👉 Final prediction = **110**

Formula in Classification

Classification Formula का मतलब

$$\hat{y} = mode(h_1(x), h_2(x), \ldots, h_n(x))$$

👉 mode = जो value सबसे ज्यादा बार आती है

Example:

3 trees prediction देते हैं:

- $h_1(x) \rightarrow$ "Spam"
- $h_2(x) \rightarrow$ "Not Spam"
- $h_3(x) \rightarrow$ "Spam"

👉 Count:

- Spam = 2
- Not Spam = 1

👉 Final output = **Spam**

✔ इसे कहते हैं **Majority Voting**

Gini Index

Gini Index क्या होता है?

👉 Gini Index बताता है:

> data कितना pure (**एक ही class का**) है

📌 **Formula:**

$$Gini = 1 - \sum p_i^2$$

👉 यहाँ:

- p_i = किसी class की probability

✔ **Gini Index**

→ data कितना pure है

→ कम Gini = अच्छा split

In Scikit-learn

Classification

Python

```python
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,
    max_depth=5,
    random_state=42
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Regression

Python

```python
from sklearn.ensemble import RandomForestRegressor

model = RandomForestRegressor(
    n_estimators=100,
    max_depth=5,
    random_state=42
)

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

Parameter with details

Parameter	क्या करता है	आसान भाषा में मतलब	कब बढ़ाएँ / घटाएँ
n_estimators	कितने trees बनेंगे	जंगल में पेड़ों की संख्या 🌳	ज्यादा trees → accuracy बेहतर (लेकिन slow)
max_depth	पेड़ की depth	पेड़ कितना गहरा जाएगा	ज्यादा depth → overfitting, कम → simple model
min_samples_split	split करने के लिए min samples	node कब टूटेगा	ज्यादा value → कम splitting → overfitting कम
min_samples_leaf	leaf node में min samples	आखिरी node में कितना data होगा	बढ़ाने से model stable होता है
max_features	कितने features use होंगे	हर decision में कितने columns देखे जाएँगे	कम रखो → overfitting कम
random_state	randomness control करता है	हर बार same result लाने के लिए	debugging और reproducibility में useful
bootstrap	sampling use करना या नहीं	data को random चुनना या पूरा use करना	True आमतौर पर बेहतर होता है

Default parameter

Default parameters क्या होते हैं?

Library पहले से कुछ values set करके रखती है, जैसे:

Python

```python
RandomForestClassifier(
    n_estimators=100,
    max_depth=None,
    min_samples_split=2,
    min_samples_leaf=1,
    max_features='sqrt',
    bootstrap=True
)
```

Metrics for Accuracy

📏 Accuracy Check करने के Metrics

🏷️ Classification Metrics

1. Accuracy
2. Precision
3. Recall
4. F1-score
5. Confusion Matrix

👉 sklearn:

```python
from sklearn.metrics import accuracy_score, classification_report

accuracy_score(y_test, y_pred)
print(classification_report(y_test, y_pred))
```

📊 Regression Metrics

1. MAE (Mean Absolute Error)
2. MSE (Mean Squared Error)
3. RMSE
4. R^2 Score

👉 sklearn:

```python
from sklearn.metrics import mean_squared_error, r2_score

mean_squared_error(y_test, y_pred)
r2_score(y_test, y_pred)
```

Summary

Short Summary

- 👉 Random Forest = Multiple Decision Trees
- 👉 Overfitting कम करता है
- 👉 Regression → Average
- 👉 Classification → Voting
- 👉 High accuracy + robust model

GridSerarhCV

- **M**